

Impact of ROS 2 Node Composition in Robotic Systems

Steve Macenski , Alberto Soragna, Michael Carroll , and Zhenpeng Ge 

Abstract—The Robot Operating System 2 (ROS 2) is the second generation of ROS representing a step forward in the robotic framework. Several new types of nodes and executor models are integral to control where, how, and when information is processed in the computational graph. This letter explores and benchmarks one of these new node types - the *Component* node - which allows nodes to be composed manually or dynamically into processes while retaining separation of concerns in a codebase for distributed development. Composition is shown to achieve a high degree of performance optimization, particularly valuable for resource-constrained systems and sensor processing pipelines, enabling distributed tasks that would not be otherwise possible in ROS 2. In this work, we briefly introduce the significance and design of node composition, then our contribution of benchmarking is provided to analyze its impact on robotic systems. Its compelling influence on performance is shown through several experiments on the latest Long Term Support (LTS) ROS 2 distribution, Humble Hawksbill.

Index Terms—Engineering for robotic systems, software architecture for robotic and automation, software tools for robot programming.

I. INTRODUCTION

THE Robot Operating System (ROS) is the largest open-source robotics ecosystem, created in 2007 by the robotics incubator Willow Garage [1]. It consists of a middleware and extensive tooling commonly required to build a variety of robotic systems. However, it was built for researchers by researchers and lacked some features necessary to build robust commercial robots.

ROS 2 was entirely redesigned to address the challenges facing the commercialization of robotics technology, while continuing its legacy of a broad ecosystem of distributed, community driven projects. It tackles long-standing problems including security, embedded and real-time support, and operating in challenging networking environments [2].

Manuscript received 2 February 2023; accepted 16 May 2023. Date of publication 24 May 2023; date of current version 30 May 2023. This letter was recommended for publication by Associate Editor Y. Hirata and Editor P. Vasseur upon evaluation of the reviewers' comments. This work was supported in part by Samsung Research, in part by iRobot, and in part by Open Robotics. (Corresponding author: Steve Macenski.)

Steve Macenski is with Samsung Research America, Mountain View, CA 94043 USA (e-mail: stevenmacenski@gmail.com).

Alberto Soragna is with iRobot Corporation, Bedford, MA 01730 USA (e-mail: asoragna@irobot.com).

Michael Carroll is with Open Robotics, Mountain View, CA 94043 USA (e-mail: michael@openrobotics.org).

Zhenpeng Ge is with the University of Electronic Science and Technology of China, Chengdu 610056, China (e-mail: gezhenpeng@std.uestc.edu.cn).

Digital Object Identifier 10.1109/LRA.2023.3279614

ROS 2 also introduces approaches enabling developers to create performant, deterministic, and safe systems. It encourages users to take control over a program's lifecycle, execution model, and resources. Two of the provided mechanisms are new types of *Nodes* and *Executors*. *Lifecycle* and *Composition* nodes allow programs to have structural control over when and where a program executes, respectively [2].

This letter will focus on the latter, the composition node, which enables a designer to manually or dynamically instantiate nodes into processes. They are also referred to more succinctly as *Components*. Composition was created to combat performance issues of one program per process while being unobtrusive to end-users [2]. Since many independent components may be assembled into a single process, ROS 2 maintains a distributed development model. Composition *dramatically* improves performance of the system from compute and memory, to latency and throughput. The impact is significant when developing resource-constrained systems, practical systems with dozens of nodes, or particularly large messages common to sensor processing pipelines. Indeed, processing pipelines with message sizes larger than 1 MB, such as images and point clouds, are virtually impossible without node composition. Further, according to iRobot and discussed in Section VI-B, products like the iRobot Create 3 would not have been possible without ROS 2 composition reducing the computational burdens on its embedded processor. Composition also allows for intra-process communication (IPC), further improving performance with zero-copy.

ROS 2 facilitates control over the execution model of the communication system. These executors are specified in a program and define how information is processed within a node. Thus, it is impossible to decouple the executor from the performance of a node - particularly when components are contained in the same process potentially sharing an executor.

In this letter, we briefly describe the concepts and design of process management in ROS 2. We also introduce for the first time the executors and their complementary component containers needed to perform dynamic composition.

The letter's contribution is extensive performance benchmarking and analysis of composition to clearly demonstrate its power and importance within the roboticists' portfolio of tools. These experiments are performed across multiple methods of composition, executor models, and communication. Additionally, an illustrative experiment is performed on a mobile robot system using the popular ROS 2 Nav2 navigation system to

reveal the significant system-wide improvements composition can make on an autonomous robot [3].

II. RELATED WORK

There has been much research interest in the performance of the ROS 2 middleware framework due to its widespread use. There have been comparisons between ROS 1 and ROS 2's performance, as well as benchmarking the performance when varying the underlying DDS implementations [4]. Several efforts have created performance measuring tools for various facets of a ROS 2 system. The *ros2_tracing* framework was introduced based on Linux Trace Toolkit Next Generation (LTTng) that allows for detailed analysis of instrumented events in a running ROS 2 system [5]. Tracing analysis is extended by adding the ability to do message flow analysis by associating causally-linked tracing events in the ROS 2 computation graph [6]. *Autoware_Perf* builds upon the *ros2_tracing* framework to add several measurements, including end-to-end latency, intra-node latency, and inter-node latency [7]. In addition, the tracing framework is evaluated against *Autoware.Auto* as a reference system, containing an architecture of nodes which simulates a real-world self-driving vehicle software scenario.

The Chain-Aware ROS Evaluation Tool (CARET) also builds upon the *ros2_tracing* framework. CARET is used to perform analysis of executor performance against the *Autoware* reference system [8], [9], but does not compare varying node architectures to characterize composition or differing communication methods (IPC, shared memory).

In contrast to direct measurement using tracepoints, latency and determinism can be considered via formal analysis of the scheduling and execution of chains of callbacks. For real-time applications, determining latency bounds is critical, so analysis to prove this is an important contribution [10]. This analysis however does not account for inter-process versus intra-process communication, which was addressed in follow-on analysis [11]. These analyses focus on worst-case latency of chains of callbacks, excluding composition and do not consider other key metrics.

In addition, there is also work in improving the performance of ROS 2 by replacing the default executor. One approach introduced a callback group executor with the aim to make ROS 2 suitable for real-time environments [12]. This executor improved latency and determinism. This work did not study the use of composition or communication methods. Another approach to improving the executor is by introducing a scheduler that is aware of the chain of callbacks to be executed [13]. This work establishes the bounds on average and worst-case latency, but does not vary the communication mechanisms available in ROS 2.

While there is literature describing some elements of ROS 2 performance, our survey compares and analyzes the composition pattern which is not explicitly addressed in previous works. None of the existing body of work evaluates detailed performance metrics of composition and its applied relevance to robotics systems researchers and designers.

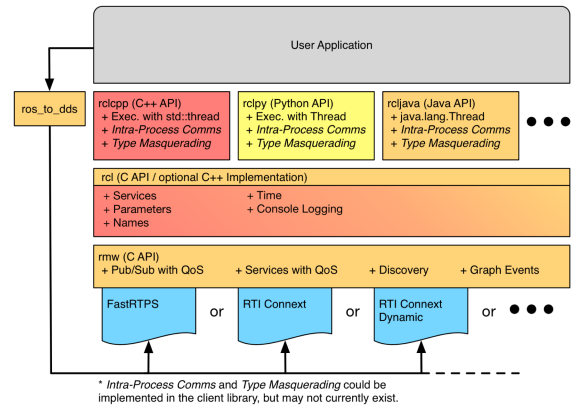


Fig. 1. ROS 2's abstraction layers including DDS, *rmw*, *rcl*, and client libraries [2].

III. REVIEW OF THE ROBOT OPERATING SYSTEM 2

ROS 2 includes all of the familiar core features of ROS including publish/subscribe, services, actions, parameters, time, logging, and more. Internally, ROS 2 is designed with modular abstraction layers to allow for use-case specific or updated integrations over time, shown in Fig. 1 [2]. The *rmw* (ROS Middleware) is the abstraction for the middleware vendor to allow for multiple communication systems to be integrated. The principal adoption of DDS for *rmw* implementations has enabled ROS 2 to provide necessary features for modern robotics development [2], [14]. Though, non-DDS *rmws* exist with community supported implementations, such as *rmw_iceoryx* enabling low latency zero-copy communication [15]. Within ROS 2, there are two key concepts that largely define the behavior of a system above the *rmw* - the Node and the Executor.

A. Nodes

A distributed system like ROS 2 can be conceptualized as a graph of end-points, each representing a process or subsystem. The graph is built from these *nodes* which act as units of processing. These can be a hardware driver, data processing, localization, and more. Each node connects to other nodes to communicate data via ROS interfaces; namely topics, services, or actions.

There exist three major types of nodes. The most basic is the Node (without qualifiers) which is the base building block for all others. The next is the Lifecycle Node, which implements a state machine for managing the program deterministically for reliable execution. A node's state may be made dependent on other nodes to create a cascading bringup whereas each node is only activated once its constituent dependencies are also activated.

Finally, the Composition Node allows for a program to be placed within processes either at compile or run-time. This contrasts with other nodes, which are created in their own processes. Composition reduces networking-related delays by using process-local optimizations and reduces overall resource consumption. This is explored in detail in Section IV. Note that these nodes are not mutually exclusive; a node can be a

lifecycle-component to leverage both deterministic execution and networking optimization.

B. Executors

The *Executor* is the mechanism by which the order and timing of available tasks are coordinated. The executor allows the communication system's execution model to be decoupled from the structure of the system graph. As network and timer events occur, they are marked in a structure called a *waitset*. The waitset is an array of entities, each indicating the status of the corresponding task. Multiple nodes or groups of callbacks can be serviced by an executor. When evaluated, an executor checks the waitset and services the next appropriate timer, topic, service, or action according to scheduling semantics, invoking relevant user-defined callbacks.

ROS 2's C++ client library currently provides three default executors. The first is a Single-threaded executor, where all incoming events are serviced in order within a single thread. A disadvantage of this executor is that any user-implemented blocking callback will also block other pending tasks.

As an alternative, the Multi-threaded executor uses a pool of threads to execute many user callbacks in parallel. This can, however, cause callbacks to be starved if others use the shared resources greedily. To refine the parallelism, interfaces can be added to a *callback group*. Each callback group can be mutually exclusive (members should not be executed in parallel) or reentrant (members may be executed in parallel).

An executor's overhead can be considerable when reconstructing the waitset every iteration. Thus, a third *Static* single-threaded executor is available. The static single-threaded executor does not reconstruct the list of event sources each iteration, but modifies an existing list as entities are added or removed.

In addition to the default set of executors, additional executors have been proposed: The *Priority-Driven Chain-Aware Scheduler* [13] improves end-to-end system latency, the *rcl Executor* [16] provides deterministic scheduling for resource limited platforms, and the *events-based executor* [17] eliminates overhead found in default implementations' waitsets.

C. Zero-Copy Communication

ROS 2 allows for improved performance through zero-copy communication using IPC and loaned messages optimizations. Both avoid the copying of data to and from the middleware layer, serialization and de-serialization, and interactions with the ROS interfaces.

IPC is used to maximize message passing performance of components in the same process, achieving similar performance to functional programming while retaining pub-sub semantics and modular architectures. When a subscription and publisher of a topic reside in the same process, the underlying middleware can be entirely short-circuited; messages are passed via smart pointers pushed into shared ring buffers [18]. IPC can be enabled through a node's options, but it presently has some implementation limitations: it is only compatible with a subset of Quality of Service (QoS) settings [14] and enabling it with subscriptions from other processes reduces its benefits.

The loaned messages API borrows memory from the middleware to perform optimizations like using shared memory buffers to enable multi-process zero-copy communication. Using loaned messages requires the use of different publisher APIs and a relatively complex, vendor-specific configuration of the DDS layer and underlying Operating System to allocate the shared memory segment. Messages published using loaned message must also have a fixed size that is known at compile-time. If subscriptions from other machines exist, it is not recommended to enable loaned messages due to double delivery of messages. Tests using loaned messages exhibit stability issues such as frequent crashes and dead-locks, discussed in further detail in Section V. In contrast to IPC, these problems are due to the currently available `rmw` implementations. For these reasons, loaned messages support is considered experimental in ROS 2 as of the time of writing. `rmw_iceoryx` is an example `rmw` implementing shared-memory IPC with message loaning [15].

IV. OVERVIEW OF ROS 2 COMPOSITION

This section is a description of key details regarding composition as it exists important for understanding the contributed benchmarking and performance analysis in Section V. Composing multiple programs into the same process saves computing resources, reduces latency, and allows for a highly distributed development workflow to be employed. However, this may not be optimal in all situations. Thus, composition allows system designers to decide which process a program should reside in at either run-time or compilation-time without software changes.

For systems under development, components are often placed in separate processes so that failures do not disturb the larger system and analysis tools can be run in a well-defined scope. As programs become more mature, they may be grouped into processes to reduce latency or share resources.

A key principle in the composition design is to be unobtrusive [2]. Unlike *nodelets* in ROS 1, components do not require any specific method implementations and it is trivial to convert a typical node into a component. With a node class derived from `Node`, the only requirement is that a constructor must take in a single argument, `NodeOptions`, containing communication and node parameterizations.

There are two mechanisms for aggregating components: manual (compile-time) or dynamic (run-time) composition. Note that either approach is available via the same API without code duplication.

A. Manual Composition

Manual composition is performed at compile time, with a user developed executable that explicitly instantiates components. Since decisions are made at compilation-time, this method provides the least flexibility, but the maximum opportunity to control resource allocation between the components. With manually-defined components in a process, the developer has the ability to explicitly control thread priorities and distribution. Any executor(s) can be used in manual composition and is specified in the program's entry point.

In general, manual composition with a single executable is most optimal for encapsulating an entire system once all exploratory development is complete and the overall system architecture is static and stable. Typically, this is leveraged in highly resource constrained environments, whereas minor overhead for flexibility is undesirable.

B. Dynamic Composition

Dynamic components are built as shared libraries and registered with a global index for bookkeeping. They are then available to be loaded into a component container, which instantiates the nodes. This can be done within a ROS 2 launch file or from a service call.

In contrast to manual composition, dynamic composition retains maximum run-time flexibility, allowing designers to distribute components across various processes at or even after launch. Distributing components allows for the grouping of related tasks in the same process, while retaining the ability to isolate components at will. All components may be loaded into a single process, like in manual composition, with little overhead.

C. Component Containers

Component containers provide a common executable for dynamically composing components, typically surrounding a particular executor. The containers provide services to add, introspect, or remove components via the component manager. A component container will search the component index and load the shared library module that contains a node of interest. If the default containers are insufficient, new component containers may be written without modifying ROS 2 itself.

There are 3 default component containers available:

- A single-threaded executor container, where components share the executor.
- A shared multi-threaded executor container, where components share the executor.
- An isolated single-threaded executor container, where each component receives its own executor.

D. Best Practices

Best practices can be summarized as the following:

- All nodes should be written as components.
- Prefer dynamic composition over manual composition when not in extreme resource constrained environments.
- With many independent components, prefer single-threaded executor(s) over one multi-threaded executor.
- Prefer one multi-threaded executor when component(s) are specifically designed to not interfere with each other.

V. BENCHMARKING AND PERFORMANCE

The effectiveness of composition is evaluated in a variety of scenarios. The following metrics were collected throughout the experiments:

- Proportional Set Size (PSS): the physical RAM usage as the sum of its private memory and the fraction of its shared memory utilized divided by the number of processes it is

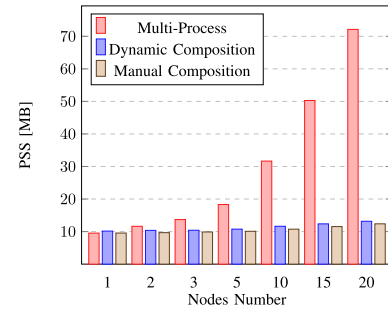


Fig. 2. RAM use for varying numbers of nodes.

shared with. This is the most accurate model as ROS 2 contains many shared libraries [19].

- CPU percentage: the ratio between wall clock time and the time spent by the CPU to execute a process.
- Latency: the time difference between message publication and a subscription callback invocation.
- Goodput: application-level throughput, the amount of published data reaching the callback per second [20].

Tests were run on Ubuntu 20.04. The Linux `proc` pseudofilesystem was used to accurately record operating-system level metrics. The communication-related metrics were computed by adding complimentary trackers in the source-code of the profiling applications. The Humble Hawksbill distribution's default configuration were used throughout the experiments unless explicitly noted. The CycloneDDS `rmw` was selected for these experiments.

All evaluations were performed on a Raspberry Pi 4 Model B, featuring an ARMv8-A 64 b processor equipped with Quad-Core Cortex-A72 1.5 GHz CPU. This embedded board was chosen because it is representative of the hardware commonly found in consumer robotics products. Using a dedicated embedded test platform also reduced noise and jitter originating from background processes that could affect results. Each experiment was conducted ten times and the measurements were averaged across consecutive runs.

We selected an existing framework¹ for reproducible evaluation of arbitrary ROS 2 systems, which we extended to support composition.

A. Memory Footprint

The first experiment evaluates the memory requirements of ROS 2 applications, comparing the two composition approaches with a standard multi-process system. Each was tested with a varying number of empty nodes to evaluate baseline memory requirements. The single-threaded executor was utilized for each test in this analysis. Note that the choice of executor does not impact memory utilization.

The results of this experiment are shown in Fig. 2 and indicate a significant improvement utilizing any of the composition approaches. The standard deviation between experimental runs was less than 1%.

¹[Online]. Available: <https://github.com/irobot-ros/ros2-performance>

Each ROS 2 process creates its own DDS network participant to communicate, which represents a considerable memory increase. The increase in memory in the multi-process system is non-linear: 2.1 MB increase on the second process that grows to 4.3 MB by the twentieth. This is due to the creation of a fully connected and distributed network graph needing to allocate resources for every discovered end-point, growing $O(N^2)$. This happens during network discovery and it is irrespective of whether the nodes will ever communicate. The consequence is that a 20 node system with composition requires approximately the same memory as a 2 node multi-process system. It is common for robot systems to contain 20 nodes or more; each hardware driver, robotic algorithm, or subsystem are typically modeled as a node. The autonomous navigation system evaluated in Section VI-A alone contains 15 nodes, not counting the drivers, autonomy application, etc needed for a complete product. Succinctly, practically complex robotic systems must use composition to stem the quadratic growth of DDS overhead.

Note that dynamically composed systems are slightly more expensive due to the additional component manager node. They also require registration with an index and linking with the component library, both of which may be omitted for manual composition. However, experiments showed a negligible increase in RAM (less than 100 KB) for twenty nodes. Thus, it is recommended to register components to take advantage of flexible dynamic composition for prototyping and testing, even when somewhat resource constrained.

B. CPU & Latency Analysis for Different Message Sizes

The next experiment evaluates the communication performance of composition. This is not only affected by how the components are organized, but also by design decisions such as the selection of executors and communication settings.

The first test measures the steady-state CPU usage and mean latency across different systems comprising two nodes: one with a publisher and another with a subscription. This experiment was run using both composition types and multi-process systems, each with a single-threaded executor. The comparison also includes a dynamically composed system with IPC and a multi-process system using loaned messages.

Each test is repeated using different message sizes. The publication frequency is set to 50 Hz to allow for appreciable differences without nearing limitations of the embedded platform for larger messages. Subscription callbacks do not perform any computation, thus all of the measurements can be attributed to communication.

The results are shown in Fig. 3 on a log-scale, showing reduction in both CPU and latency by more than half for every message size with either type of composition when compared to the standard multi-process approach. Without zero-copy, performance degrades proportional to the message size.

Multi-process communication without loaned messages has a latency exceeding 30% of the publication period (a key performance indicator) for messages above 1 MB. Enabling loaned messages shows a near constant performance regardless of the message size. This displays the highest overhead for small

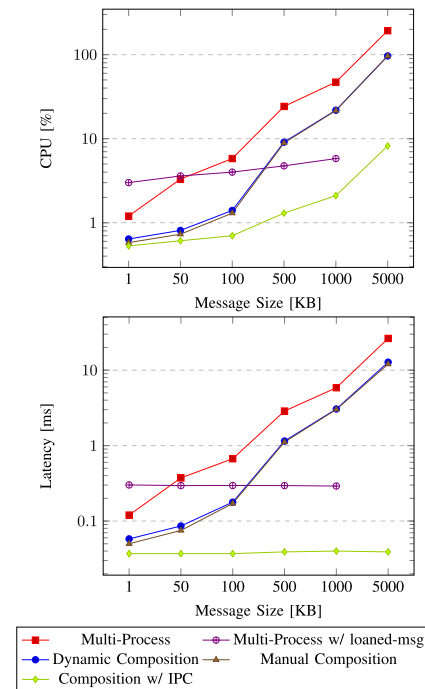


Fig. 3. CPU, latency for different message sizes.

messages, but later demonstrates a major improvement for the larger messages - although overall remaining less efficient than IPC. The loaned messages optimization showed problems when using messages larger than 2 Mb, which caused the application to repeatedly crash. For this reason, this is not available in Fig. 3.

Each of the composition methods without IPC have remarkably similar performance; nevertheless the manual approach is slightly better due to the absence of the component manager. Although this node does not process work at steady-state, the single-threaded executor is required to check it, adding a minor amount of overhead. Indeed, the performance difference between the composition methods is only perceptible for the smallest message sizes where the manager contributes a larger proportion of the work completed.²

IPC in dynamic composition displayed a constant latency of 40 μ s throughout all of the experiments, without scaling by message size. CPU is reduced by almost an order of magnitude for larger messages; though it scales by message size due to the time it takes to populate the messages themselves. Composition and IPC allow ROS 2 systems to achieve negligible latency, similar to monolithic applications. Without Composition with IPC, ROS 2 could not support extreme resource constrained systems.

C. Maximum Achievable Goodput

The next experiment computes the maximum amount of data that can be transferred between two entities. The system is made up of a publisher and a subscription node, where we measure

²Later experiments show the ‘worst-case’ (dynamic) due to the very small performance differences, adding clarity to later figures. Manual composition experiments were performed and exhibited analogous behavior.

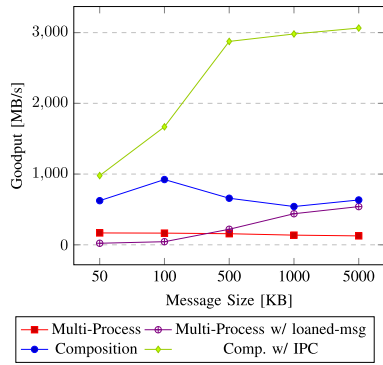


Fig. 4. Maximum goodput between one publisher and one subscription for different message sizes.

the goodput by publishing continuously without pause using a single-threaded executor. This metric measures only the amount of useful data received by the subscription callback, excluding additional payloads used by the `rwm` for the transmission. The results for variable message sizes are shown in Fig. 4, where single-process solutions can reach a far higher maximum rate.

There is consistent overhead independent of data size from the executor and data handling deep within ROS 2. For this reason, smaller messages have a relatively low goodput as these fixed costs reduce the maximum rate at which a subscription can receive data.

Multi-process without loaned messages reaches its peak for messages slightly smaller than 65 KB, which corresponds to the maximum UDP datagram length [21]. When the payload exceeds that, the DDS middleware fragments the message into multiple packets for transport, with a negative impact on performance. Although the multi-process approach transmits the least amount of data, it uses the most CPU, slightly above 200%. By comparison, a single-process approach used 82%.

Using loaned messages with multi-process communication resulted in deadlocks and other stability issues when the publication frequency reached 500 Hz. This required artificially limiting the maximum rate to 450 Hz to obtain valid data and is why loaned messages underperformed when publishing small payloads. The 1000 KB and 5000 KB tests did not reach the 450 Hz cap, with maximum frequencies 430 Hz and 110 Hz respectively. This allowed appreciable performance improvements to be seen while using loaned messages in the multi-process scenario at these message sizes - though still far slower than IPC.

Enabling IPC yields again the best results. The goodput increases with message size, peaking due to the publisher's message allocation speed limit.

Our analysis shows that composition with IPC is a requirement for applications that must produce data at very high frequencies, such as above 1 KHz. Many SLAM and Visual Odometry systems require images at 30+ Hz: using a VGA three channel image, multi-process cannot communicate an uncompressed feed at 20 Hz, while composition and IPC can stream at 85 Hz and 415 Hz, respectively. Multi-process solutions with loaned messages can achieve the required frame-rate, but with a

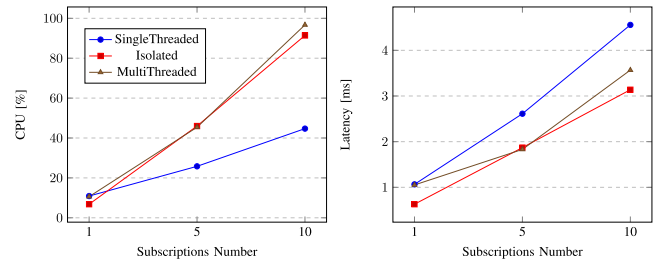


Fig. 5. CPU, latency of single-process systems with varying executors. Subscriptions callbacks do 500 μ s of work.

higher CPU usage, added complexity and, as of today, with the risk of stability issues.

D. Performance Impact of Component Containers

The component containers presented in Section IV-C play a fundamental role in the performance of a system via different executor patterns. In this last experiment, we compared the container's performance using a simple dynamic composition application. Each system comprises one publisher node, publishing messages of 500 KB at 50 Hz with a variable number of subscriptions. Different from the previous tests, computations of 500 μ s were added to each subscription callback to emulate data-processing (Fig. 5).

The single-threaded pattern requires the least CPU because the pipeline needs to interact with the `rwm` only once per iteration, regardless of the number of subscriptions. However, it also displays the highest latency since callbacks cannot execute in parallel. Interestingly, when removing work from the callbacks, this approach actually becomes the fastest since the sequential execution of callbacks is outweighed by the reduced `rwm` overhead.

The isolated and multi-threaded containers have similar performance, but with worthwhile differences. The multi-threaded container spawns the maximum number of concurrent threads supported by the architecture, which are not tied to any specific interface. With only one subscription, parallelization is not possible, so the performance of the multi- and single-threaded containers are essentially identical.

The performance of the isolated and multi-threaded containers are similar as the number of subscriptions increases, with parallelization allowing to achieve lower latency than purely sequential execution. The isolated container however yields better performance after the number of subscriptions exceeds the number of threads in the multi-threaded container (e.g. number of cores). Additionally, the isolated container achieves the best performance when ROS 2's timers are used for periodic tasks. Timers must be checked at every iteration of the executor, so by using dedicated executors for each node, the overhead of a timer applies only to the executor which contains it.

This experiment highlights that executors behave differently depending on the topology of the system and the constituent node's particular implementations. In general, the isolated container provides a reasonable default when composing systems of non-trivial nodes. Application developers are encouraged to

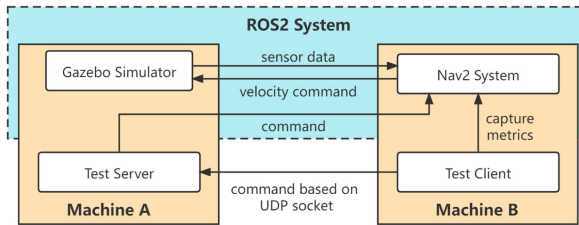


Fig. 6. Testing framework for Nav2 system evaluation.

explore the different modes and develop their own containers with more advanced configurations, such as specifying different types of executors for the different nodes or thread priorities and CPU affinities.

VI. BENCHMARKING ON REALISTIC ROBOTIC SYSTEMS

Additional experiments were performed to showcase the practical impacts that ROS 2 composition has on autonomous robotics systems. The previous experiments clearly demonstrates the trends and behaviors of composition at a benchmarking level, but they do not illustrate the impact that could be expected on a full system when exploited. Thus, this subsection is designed to briefly highlight the impacts with and without use of composition on full systems.

A. ROS 2 Navigation

The Nav2 project is used as the autonomous robotics system for analysis [3]. It consists of a configurable and extensible behavior-tree navigator with several modular servers to accomplish tasks such as planning, control, recovery, path smoothing, route following, and more. Each of these servers is modeled as an independent component-lifecycle node that can be placed dynamically throughout the computation network. Nav2 is an ideal candidate for evaluation, with 15 nodes following the best practices in ROS 2 style, API, and design.

This experiment captures the same computation and memory metrics on the same hardware platform as presented in Section V. The navigation system is tested in the following configurations: multi-process, manual composition, and dynamic composition. Both composition methods used the isolated single-threaded executor.

In these experiments, the navigation system was executed with a simulated differential-drive robot in Gazebo and measurements were taken during steady-state navigation between two arbitrary goal points on a map generated using SLAM Toolbox with sensor processing for dynamic obstacles [22], [23], [24]. It uses the NavFn global path planner alongside the DWB local trajectory planner processing inputs from an RGB-D sensor and a 2D planar laser scanner to represent a sandbox environment. The experimental setup and its configuration are identical to that which would be used in practical service robotics applications, which can be found in the `nav2_bringup` package.

The testing scripts and simulator were run on a separate host computer to remove the impact of shared library components on PSS memory metrics, with setup in Fig. 6. The results of this

TABLE I
NAV2 RESOURCES USED ON ARM AND X86 PLATFORMS

ARM	PSS [MB]	CPU [%]
Multi-Process	116.63 ± 0.40	154.27 ± 3.91
Manual Composition	77.84 ± 0.47	110.50 ± 2.87
Dynamic Comp. (isolated)	78.63 ± 0.17	109.62 ± 2.43
Dynamic Comp. (multi-threaded)	75.52 ± 0.71	140.32 ± 7.05
x86	PSS [MB]	CPU [%]
Multi-Process	118.85 ± 0.36	48.60 ± 3.15
Manual Composition	67.13 ± 0.18	36.60 ± 4.22
Dynamic Comp. (isolated)	67.67 ± 0.14	36.09 ± 4.00
Dynamic Comp. (multi-threaded)	66.71 ± 0.27	46.27 ± 2.80

experiment are shown in Table I representing the average of 10 trials.

Manual and dynamic composition on the embedded ARM processor both saved an astounding 28% CPU and 33% RAM over multi-process. This is a resounding improvement in performance displayed by a full system containing advanced algorithms by composing them into a single process. While both component containers consumed relatively similar memory, the multi-threaded container consumed far more CPU - on par with full-blown multi-process communication. This is due to the use of more nodes than CPU cores.

A complementary experiment was run using an x86 platform (Intel i5-8300H) to highlight any differences due to compute architecture. In that experiment, composition saved 25% CPU and 44% RAM over a multi-process system architecture - displaying equivalent trends on both major hardware platforms.

These are significant improvements in performance due to only changes in the networking characteristics of an entire autonomous mobile robot navigation system. While continuing to plan, control, and perceive the environment, a robot has more resources to compute additional tasks such as its autonomy system or deploy more advanced perception techniques with the liberated memory and CPU time. Optionally, the freed memory resources could be used to store increasingly larger maps (40 MB can store an additional 100,000+ m^2) and freed compute time to plan longer global paths - enabling high quality navigation in larger scale spaces. Over 10% of an x86 core and 40% of an ARM core are made available again for additional tasks which would not have otherwise been possible. Since navigation is only one system on a robot, it is essential to optimize its performance to leave room for other necessary subsystems (scheduler, external communication, error checking, payload application, data management, etc).

B. Irobot Create 3

The iRobot Create 3 is a robotic platform designed to give developers access to a robust mobile base through standard ROS 2 APIs. It is equipped with a variety of sensors and actuators: encoders, optical flow and IMU for pose estimation, bumpers, cliffs and IR for obstacle detection, and wheels, LED lights and audio speakers controllable by the user.

The robot produces 70 KBps of raw sensor data and processed information, while also running a 50 Hz motion control loop and an obstacle detection pipeline. Users can either publish

individual actuation commands or send action goals to execute autonomous behaviors such as pose regulation, wall-following, and docking.

The single-board computer that is present on the robot imposed severe resource constraints to the design of the application, due to a processor with limited CPU and less than 60 MB of RAM. The robot runs a single-process, manually composed, ROS 2 application which takes advantage of IPC and an optimized executor [17]. This ROS 2 system is made of approximately 10 nodes, with more than 30 topics, 10 services and 10 action servers, in addition to the automatically created entities. ROS 2 is used both for the internal implementation of algorithms and drivers, as well as for interfacing with the user.

The iRobot Create 3 application is normally controlled by an external navigation software, such as Nav2 described in Section VI-A, and this results in the use of approximately 60% CPU and 32 MB RAM. More performance intensive operations such as remotely subscribing to all the published topics (e.g. for visualization purposes or to record a log), can increase the CPU usage to 80%. The robot also supports multiple rwm, and with some of them the RAM usage grows up to 40 MB.

By taking into account the current resource usage and the results presented in the previous experiments, it is evident a multi-process solution wouldn't have been possible. Indeed, in Fig. 2 it is shown how a 10 nodes multi-process system required roughly 3 times the RAM of a single-process, composed, one - far exceeding the embedded compute platform's resources. The run-time overhead of a multi-process configuration would have also caused consistent thread starvation and a CPU load average [25] above the number of available cores, making it impossible to achieve desired rates and smooth robot actions. Without access to ROS 2 composition, the software architecture of this robot would have been vastly different, with a non-ROS monolithic application and a single ROS node acting as the bridge with the outside world.

When creating consumer products there is often a trade-off between cost and value. The use of ROS does not directly add value to iRobot's customers; its main benefit is during development. Thanks to ROS 2 composition, it is possible to deploy ROS-based applications on low-cost embedded platforms, as exemplified by the iRobot Create 3.

VII. CONCLUSION

This letter presents how components and executors can highly impact ROS 2's performance with little overhead to a system designer. Composition enables not previously possible applications of ROS 2 in extreme resource constrained systems and sensor processing pipelines while improving performance across complex robotic systems. We showed experiments that evaluate these improvements and conclude that using any form of composition significantly improves memory, latency, and CPU usage. When IPC is enabled, it further decreases CPU usage by an order of magnitude and introduces low levels of stable latency. These performance trends lead us to recommend composition as a best practice.

REFERENCES

- [1] M. Quigley et al., "ROS: An open-source robot operating system," in *Proc. IEEE Int. Conf. Robot. Automat. Workshop Open Source Softw.*, 2009, Art. no. 5.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, 2022, Art. no. eabm6074.
- [3] S. Macenski, F. Martin, R. White, and J. Ginés Clavero, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 2718–2725.
- [4] Y. Maruyama, S. Kato, and T. Azumi, "Exploring the performance of ROS2," in *Proc. 13th Int. Conf. Embedded Softw.*, 2016, pp. 1–10.
- [5] C. Bédard, I. Lütkebohle, and M. Dagenais, "ros2_tracing: Multipurpose low-overhead framework for real-time tracing of ROS 2," *IEEE Robot. Automat. Lett.*, vol. 7, no. 3, pp. 6511–6518, Jul. 2022.
- [6] C. Bédard, P.-Y. Lajoie, G. Beltrame, and M. Dagenais, "Message flow analysis with complex causal links for distributed ROS 2 systems," *Robot. Auton. Syst.*, vol. 161, 2023, Art. no. 104361.
- [7] Z. Li, A. Hasegawa, and T. Azumi, "Autoware_perf: A tracing and performance analysis framework for ROS 2 applications," *J. Syst. Architecture*, vol. 123, 2022, Art. no. 102341. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1383762121002344>
- [8] B. Peng, A. Hasegawa, and T. Azumi, "Scheduling performance evaluation framework for ROS 2 applications," in *Proc. IEEE 24th Int Conf High Perform. Comput. Commun.; 8th Int Conf Data Sci. Syst.; 20th Int Conf Smart City; 8th Int Conf Dependability Sensor, Cloud, Big Data Syst. Appl.*, 2022, pp. 2031–2038.
- [9] T. Kuboichi et al., "CARET: Chain-aware ROS 2 evaluation tool," in *Proc. IEEE Int. Conf. Embedded Ubiquitous Comput.*, 2022, pp. 1–8.
- [10] D. Casini, T. Blaß, I. Lütkebohle, and B. Brandenburg, "Response-time analysis of ROS 2 processing chains under reservation-based scheduling," in *Proc. 31st Euromicro Conf. Real-Time Syst.*, 2019, pp. 1–23.
- [11] X. Jiang, D. Ji, N. Guan, R. Li, Y. Tang, and Y. Wang, "Real-time scheduling and analysis of processing chains on multi-threaded executor in ROS 2," in *Proc. IEEE Real-Time Syst. Symp.*, 2022, pp. 27–39.
- [12] Y. Yang and T. Azumi, "Exploring real-time executor on ROS 2," in *Proc. IEEE Int. Conf. Embedded Softw. Syst.*, 2020, pp. 1–8.
- [13] H. Choi, Y. Xiang, and H. Kim, "PiCAS: New design of priority-driven chain-aware scheduling for ROS2," in *Proc. IEEE 27th Real-Time Embedded Technol. Appl. Symp.*, 2021, pp. 251–263.
- [14] Object Management Group, "Data distribution service for real-time systems specification," Dec. 2004.
- [15] K. Knese and M. Pöhl, "A true zero-copy RMW implementation for ROS 2," in *ROSCon*, 2019.
- [16] J. Staschulat, I. Lütkebohle, and R. Lange, "The RCLC executor: Domain-specific deterministic scheduling mechanisms for ROS applications on microcontrollers: Work-in-progress," in *Proc. Int. Conf. Embedded Softw.*, 2020, pp. 18–19.
- [17] "Events executor design," Accessed on: Aug. 22, 2022. [Online]. Available: <https://github.com/ros2/design/pull/305>
- [18] A. Soragna, J. Oxoby, and D. Goel, "ROS2 for consumer robotics," in *ROSCon*, 2019.
- [19] M. Mackall, "System-wide memory profiling," in *Proc. CELF Embedded Linux Conf.*, 2007.
- [20] D. Comer, *Computer Networks and Internets*, 6th ed. Indianapolis, IN, USA: Pearson IT Certification, 2018.
- [21] K. R. Fall and W. R. Stevens, *TCP/IP Illustrated, The Protocols*, vol. 1. Reading, MA, USA: Addison-Wesley, 2011.
- [22] S. Macenski and I. Jambrecic, "SLAM toolbox: SLAM for the dynamic world," *J. Open Source Softw.*, vol. 6, no. 61, 2021, Art. no. 2783.
- [23] S. Macenski, D. Tsai, and M. Feinberg, "Spatio-temporal voxel layer: A view on robot perception for the dynamic world," *Int. J. Adv. Robot. Syst.*, vol. 17, no. 2, 2020, Art. no. 1729881420910530.
- [24] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2004, vol. 3, pp. 2149–2154.
- [25] R. Walker, "Examining load average," *Linux J.*, vol. 2006, no. 152, 2006, Art. no. 5.